Trabalho Prático: Divisão e Conquista

Disciplina: Construção de Algoritmos **Integrantes:** Cairo e Valter

Objetivo. Comparar soluções de força bruta e divisão e conquista em dois problemas: subarranjo de soma máxima e par de pontos mais próximos. Também implementei uma versão linear para o primeiro problema, conforme o desafio citado na avaliação.

1. Subarranjo de soma máxima

O problema recebe uma sequência de números positivos e negativos e procura o intervalo contínuo com maior soma. Os índices mostrados no programa começam em zero. Quando há empate, escolho o menor índice inicial e, depois, o menor índice final. Um vetor só com números negativos devolve o maior elemento, sem considerar o intervalo vazio.

Força bruta. Para cada posição de início, percorro todos os finais possíveis, atualizando a soma. Assim, evito somar o mesmo intervalo novamente do zero. Custo: $O(n^2)$ de tempo e O(1) de memória auxiliar.

Divisão e conquista. Divido o vetor ao meio, resolvo recursivamente as metades e calculo o melhor intervalo que cruza o meio, fazendo uma varredura para a esquerda e outra para a direita. O resultado é o melhor entre os três. Recorrência: $\mathbf{T(n) = 2T(n/2) + \Theta(n)}$; pelo Teorema Mestre, $\mathbf{T(n) = \Theta(n \log n)}$. A profundidade da recursão é O(log n).

Desafio linear. Usei a ideia de Kadane: em cada posição, decido entre continuar a soma anterior ou começar um novo intervalo. Cada elemento é visitado uma vez: $\Theta\mathbf{(n)}$ de tempo e O(1) de memória auxiliar.

2. Par de pontos mais próximos

A entrada é um conjunto de estações com coordenadas (x, y). A distância usada é a euclidiana. Durante as comparações calculo a distância ao quadrado, para evitar raízes desnecessárias; a raiz é feita apenas no resultado.

Força bruta. Comparo cada ponto com todos os pontos seguintes. São $n(n–1)/2$ pares, portanto $\Theta\mathbf{(n^2)}$ de tempo.

Divisão e conquista. Ordeno os pontos por x e por y. Divido pelo meio no eixo x, resolvo os dois lados, e uso a menor distância δ para montar uma faixa em torno da divisão. Como a faixa já está ordenada por y, para cada ponto testo no máximo os 7 seguintes. Recorrência: $\mathbf{T(n) = 2T(n/2) + \Theta(n)}$, após a ordenação inicial O(n log n); custo total $\Theta\mathbf{(n \log n)}$.

Por que 7 vizinhos bastam? Como não existe um par dentro de cada metade com distância menor que δ, um argumento geométrico de empacotamento limita a quantidade de pontos próximos de um ponto na faixa. Ao considerar um retângulo de altura δ e largura 2δ e subdividi-lo em regiões menores, não cabem mais de 7 candidatos seguintes em ordem de y que precisem ser testados para achar uma distância estritamente menor. Dessa forma, a combinação continua linear no tamanho da faixa. Em empates, pode existir mais de um par correto.

Limiar de 32 pontos. Para subproblemas com até 32 pontos usei força bruta: nesses casos o custo quadrático ainda é pequeno e evito criar mais chamadas recursivas. O valor 32 é uma escolha prática, não um ótimo comprovado; para escolher o melhor valor com segurança, seria necessário comparar vários limiares na mesma máquina.

3. Testes realizados

Executei a classe Testes.java e obtive **1.076 verificações aprovadas**. Os casos fixos incluem vetor só negativo, tamanho 1, zeros, empate entre intervalos, mistura de sinais, dois pontos, pontos repetidos, coordenadas negativas e um par atravessando a linha de divisão. Além disso, comparei as implementações em 400 vetores aleatórios e 250 conjuntos de pontos aleatórios, todos com semente fixa (2026).

Os testes de pontos comparam a **distância mínima**, já que vários pares podem ter a mesma distância. Para o problema de soma, comparei soma e índices, usando a mesma regra de desempate.

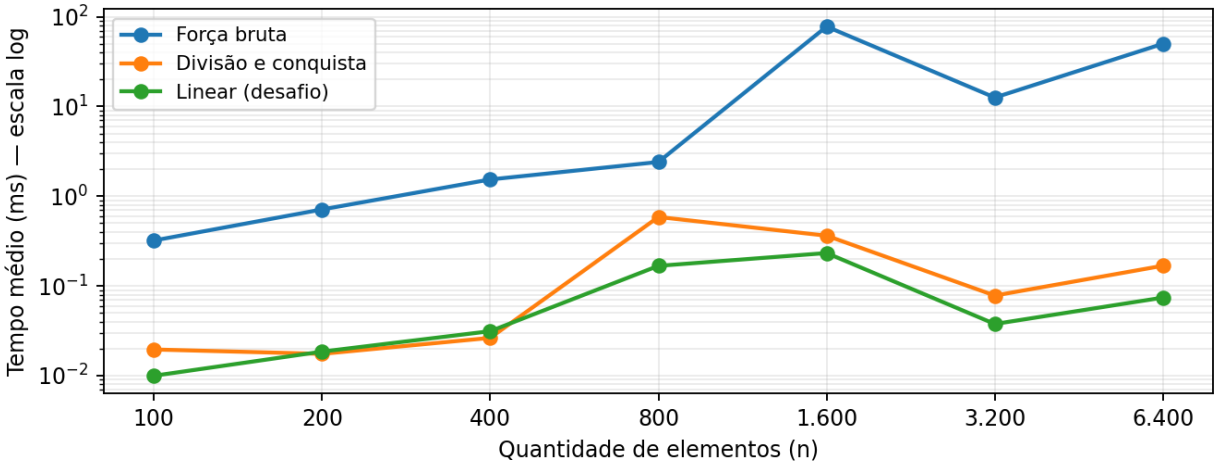
4. Como foram feitas as medições

Medi o tempo com System.nanoTime(), usando **7 execuções por tamanho e método**, depois de 8 execuções de aquecimento. Os valores brutos estão em resultados/medicoes.csv. Cada algoritmo foi medido com os mesmos dados para o respectivo tamanho. Para a soma usei inteiros aleatórios entre −1000 e 1000; para pontos usei coordenadas reais no intervalo [0, 10000). Os resultados abaixo são médias, convertidas de nanossegundos para milissegundos.

Ambiente de execução desta medição: OpenJDK 21.0.12, Linux x86_64, CPU virtual AMD EPYC 9V74 e aproximadamente 5,8 GiB de RAM disponíveis ao ambiente. Essas medições foram feitas em um ambiente de execução de teste, não no computador dos integrantes; por isso os números podem mudar quando o programa for executado em outra máquina.

5. Resultados: subarranjo de soma máxima

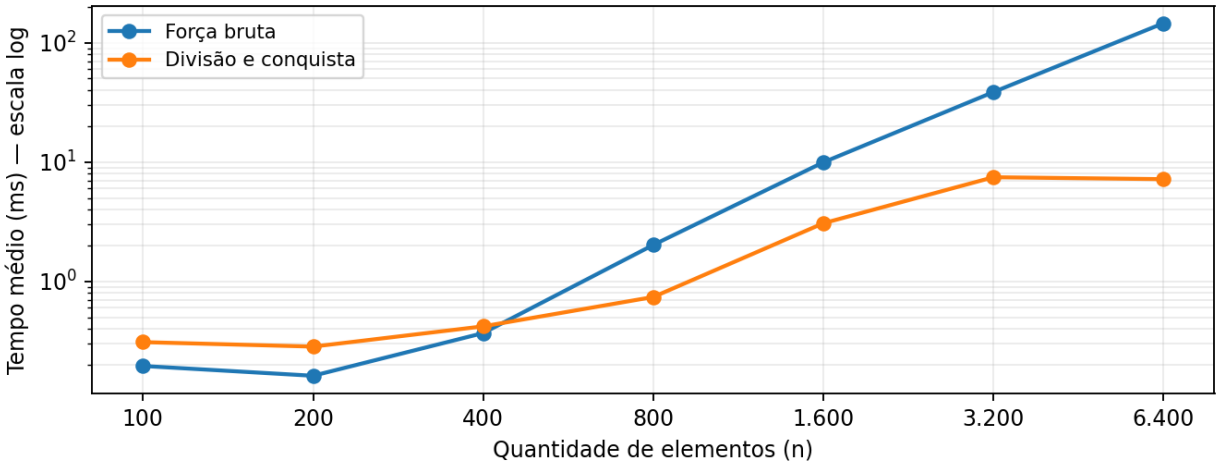
n	Força bruta (ms)	Divisão e conquista (ms)	Linear (desafio) (ms)
100	0.3210	0.0196	0.0100
200	0.7124	0.0175	0.0185
400	1.5385	0.0264	0.0313
800	2.4088	0.5872	0.1675
1600	77.8881	0.3633	0.2328
3200	12.5963	0.0782	0.0377
6400	50.1071	0.1680	0.0742



No subarranjo, a força bruta cresce muito mais rapidamente em geral, enquanto a versão linear tende a ganhar e a recursiva fica entre as duas. Alguns pontos fogem da tendência, especialmente por variações do ambiente e otimizações da JVM. Portanto, o gráfico ilustra a tendência, mas uma única medição não prova a complexidade.

6. Resultados: par de pontos mais próximos

n	Força bruta (ms)	Divisão e conquista (ms)
100	0.1954	0.3096
200	0.1617	0.2840
400	0.3685	0.4197
800	2.0180	0.7392
1600	9.9394	3.0673
3200	38.5344	7.4555
6400	145.6134	7.1844



Nos pontos, a força bruta foi mais rápida nos tamanhos menores (100 a 400), porque tem menos estrutura e não precisa ordenar nem dividir. A partir de 800 pontos, a implementação recursiva foi mais rápida nas medições. Em n=6.400, a força bruta levou aproximadamente 145,61 ms e a divisão 7,18 ms.

7. Discussão dos resultados e limites do experimento

Ao comparar os resultados, é importante não olhar apenas para o menor tempo em uma única linha da tabela. A análise de complexidade descreve a tendência de crescimento quando n aumenta, enquanto os valores medidos também incluem custos fixos, alocação de objetos, execução da máquina virtual e possíveis mudanças na carga do computador. Por isso, as curvas não precisam crescer exatamente na mesma proporção em todas as entradas.

No problema da soma máxima, o método de força bruta usa dois laços e calcula todos os intervalos possíveis. Quando dobramos n, a quantidade de intervalos cresce aproximadamente quatro vezes. Já o método recursivo divide o vetor em duas partes e faz apenas uma passagem para combinar as respostas; seu crescimento esperado é próximo de n log n. Na solução linear, basta manter a melhor soma que termina na posição atual, o que diminui bastante o trabalho realizado.

No problema dos pontos, o cálculo direto compara cada par apenas uma vez. Ele pode ser uma boa opção para poucos pontos porque não precisa criar subconjuntos, ordenar ou organizar a faixa central. Na solução recursiva, a ordenação inicial e a divisão trazem um custo adicional; em compensação, evitamos comparar a maioria dos pares quando a entrada fica grande. O limiar de 32 pontos foi escolhido para aproveitar a simplicidade da força bruta nos menores grupos.

A ideia dos sete vizinhos na faixa não significa procurar os sete pontos mais próximos em todo o plano. Depois da ordenação por y, verificamos somente os próximos candidatos na própria faixa, porque os pontos mais distantes verticalmente do que a melhor distância atual não podem melhorar a resposta. A limitação geométrica é o que impede a combinação de voltar a ser quadrática.

8. Cuidados com os testes e possíveis melhorias

Além de testar números aleatórios, foi necessário incluir situações que costumam revelar erros: listas formadas só por valores negativos, empates e subarranjos de um elemento. Nos pontos, coordenadas repetidas devem produzir distância zero; também incluí valores negativos e pares que ficam em lados opostos da linha de divisão. Esses testes são importantes porque um caso simples, sozinho, não verifica todas as decisões do algoritmo.

Uma melhoria possível seria executar as medições mais vezes e repetir o procedimento em diferentes dias, usando sempre a mesma máquina e a mesma versão do Java. Outra seria avaliar limiares como 16, 32 e 64 pontos antes de escolher o mais eficiente. Não realizei essa comparação de limiares, então não seria correto afirmar que 32 é o melhor valor em qualquer computador.

9. Conclusão

O experimento mostrou que escolher o algoritmo depende do tamanho da entrada. Uma solução simples pode vencer em casos pequenos, mas a divisão e conquista costuma escalar melhor conforme o número de elementos aumenta. No subarranjo, a versão linear do desafio é ainda mais eficiente. Aprendi também que testes de borda são essenciais: um algoritmo pode funcionar com exemplos comuns e ainda errar em números negativos, empates ou coordenadas repetidas.

Como os tempos de programas Java variam com aquecimento da JVM, coleta de lixo e carga do processador, o ideal é repetir o experimento em uma máquina fixa, com mais rodadas, se o objetivo for uma comparação mais precisa.